

Butterfly habitat layers and species habitat maps

Contents

1	Data overview	2
1.1	Conditions of use	2
1.2	Folder structure of the data delivery	2
1.3	Description of delivered files	3
1.4	Table: overview of all delivered raster files	4
1.5	Table: overview of patch-distance outputs	4
2	Overview	5
3	Packages	6
3.1	Data and folder structure	6
4	Input data	6
4.1	Majority expert opinions	7
4.2	Habitat-basemap mapping (Classes_Butterflies)	7
4.3	How the two tables interact	8
5	Expert opinions and basemap	8
6	Map Excel habitat classes to basemap classes (narrow & broad)	9
6.1	Build habitat masks (COGs) per habitat (narrow & broad)	11
7	EcoDes structural layers: vegetation density and canopy height	13
8	Forest edge distance and 200 m forest border	15
9	Forest gaps: canopy < 2 m inside forest holes > 400 m²	17
10	Distance-to-coast masks (200, 400, 800 m)	19
11	Species-specific habitat rasters (narrow & broad, with structural layers)	20
11.1	Set up structural flags per habitat	20
11.2	Build species rasters (narrow + broad)	21

1 Data overview

All paths below are given relative to the `outputs/` folder used in this project.

1.1 Conditions of use

Description: This folder contains derived spatial data on butterfly habitats in Denmark, produced as part of the projects

- **Project 1 – Contemporary Habitat Configuration and Population Genomics**, and
- **Project 2 – Historical / Temporal Habitat Configuration and Population Museomics**.

Date created: 07-12-2025

Date modified: 07-12-2025

Contact: Derek Corcoran; Signe Normand

Contact email: derek.corcoran@bio.au.dk; signe.normand@bio.au.dk

Please read and adhere to the conditions of use carefully before using these data.

Conditions of use

The data provided here must not be shared, published or otherwise used beyond the agreed collaboration without prior approval from Prof. Signe Normand (signe.normand@bio.au.dk).

- Upon agreement as stated above, the data can only be shared under a restrictive **CC BY-NC-ND** license (Attribution–NonCommercial–NoDerivatives).
- The data have been assembled specifically to support the above projects and should only be used accordingly and in agreement with the data providers (SustainScapes group and collaborators).

1.2 Folder structure of the data delivery

The delivery currently consists of:

- **SpeciesHabs/**
Species-specific habitat suitability rasters for the butterfly species included in this project (one file per species and definition).
- **Species_PatchDistances/**
Patch-based connectivity outputs (thresholded rasters, patch polygons, and nearest-neighbour distance statistics) derived from the species habitat rasters.

The **upstream input data** used to derive these layers (national land-use basemap, EcoDes layers, expert opinion tables, etc.) are *not* part of this delivery, but are fully documented in the workflow sections.

1.3 Description of delivered files

1.3.1 SpeciesHabs/ – species habitat suitability rasters

Each file in SpeciesHabs/ is a **stepped (four-level) habitat suitability index** for a single butterfly species, derived by combining the relevant habitat masks with species–habitat expert weights.

- Format: single-band GeoTIFF (Cloud Optimized GeoTIFF, COG).
- Grid / resolution: 10×10 m.
- CRS: UTM32_EUREF89, identical to the basemap (lu_00_2021.tif) and the rasters in Habitats/.

Pixel values

Values are discrete and take only the following levels:

- 0 = not considered habitat for that species
- 0.3 = poor / low-quality habitat
- 0.7 = medium-quality habitat
- 1 = high-quality habitat
- NA = outside Denmark / outside the analysis domain

These levels come from the expert “Majority” scores (0–3) in `Majority_Expert_Opinions.xlsx`, after scaling to the 0–1 range (Majority 1 $\rightarrow$ 0.3, Majority 2 $\rightarrow$ 0.7, Majority 3 $\rightarrow$ 1.0; Majority 0 is excluded and effectively coded as 0).

How the per-pixel value is constructed

For each species and pixel, we:

1. Identify all conceptual habitats that are present in that pixel (habitat mask == 1) in the chosen definition (**narrow** or **broad**).
2. For each of those habitats, multiply the binary mask by the species–habitat weight (one of 0.3, 0.7, 1.0).
3. Assign to the pixel the **maximum** of these weighted values.
If no contributing habitat is present, the pixel is set to 0.

Thus, the species rasters are **not probabilities**, but an ordinal four-level suitability index on the 0–1 scale, with possible values restricted to $\{0, 0.3, 0.7, 1\}$.

Naming convention

- <species_name>_narrow_.tif
- <species_name>_broad_.tif

where <species_name> is a cleaned version of the Latin species name, e.g. `aglais_urticae` for *Aglais urticae*.

For example:

- `SpeciesHabs/aglais_urticae_narrow.tif`
Narrow habitat suitability map for *Aglais urticae*.
- `SpeciesHabs/aglais_urticae_broad.tif`
Broad habitat suitability map for *Aglais urticae*.

1.3.2 Species_PatchDistances/ – patch connectivity outputs

The folder `Species_PatchDistances/` contains, for each species, habitat definition and threshold level:

- Thresholded rasters (`*_High.tif`, `*_Medium.tif`, `*_Low.tif`),
- Patch polygons (`*.shp` etc.), and
- CSV files with summary statistics on nearest-neighbour distances (mean, median and number of patches).

These files are described in more detail in the section “*Patch-based connectivity metrics from species rasters*”.

1.4 Table: overview of all delivered raster files

The following chunk generates a table with one row per raster file in `SpeciesHabs/`. This makes it easy for data users to see exactly what is being delivered, and it ensures that the individual file names appear in the PDF (and are searchable). See Table 1 for an overview of all delivered rasters.

Table 1: Overview of delivered raster files in `SpeciesHabs/`.

File	Name	Definition	Description
<code>aglais_urticae_broad.tif</code>	<code>aglais_urticae</code>	broad	Species-specific habitat suitability index (0–1)
<code>aglais_urticae_narrow.tif</code>	<code>aglais_urticae</code>	narrow	Species-specific habitat suitability index (0–1)
<code>coenonympha_tullia_broad.tif</code>	<code>coenonympha_tullia</code>	broad	Species-specific habitat suitability index (0–1)
<code>coenonympha_tullia_narrow.tif</code>	<code>coenonympha_tullia</code>	narrow	Species-specific habitat suitability index (0–1)
<code>gonepteryx_rhamni_broad.tif</code>	<code>gonepteryx_rhamni</code>	broad	Species-specific habitat suitability index (0–1)
<code>gonepteryx_rhamni_narrow.tif</code>	<code>gonepteryx_rhamni</code>	narrow	Species-specific habitat suitability index (0–1)
<code>lycaena_virgaureae_broad.tif</code>	<code>lycaena_virgaureae</code>	broad	Species-specific habitat suitability index (0–1)
<code>lycaena_virgaureae_narrow.tif</code>	<code>lycaena_virgaureae</code>	narrow	Species-specific habitat suitability index (0–1)
<code>maniola_jurtina_broad.tif</code>	<code>maniola_jurtina</code>	broad	Species-specific habitat suitability index (0–1)
<code>maniola_jurtina_narrow.tif</code>	<code>maniola_jurtina</code>	narrow	Species-specific habitat suitability index (0–1)
<code>pieris_napi_broad.tif</code>	<code>pieris_napi</code>	broad	Species-specific habitat suitability index (0–1)
<code>pieris_napi_narrow.tif</code>	<code>pieris_napi</code>	narrow	Species-specific habitat suitability index (0–1)
<code>thymelicus_lineola_broad.tif</code>	<code>thymelicus_lineola</code>	broad	Species-specific habitat suitability index (0–1)
<code>thymelicus_lineola_narrow.tif</code>	<code>thymelicus_lineola</code>	narrow	Species-specific habitat suitability index (0–1)

1.5 Table: overview of patch-distance outputs

The folder `Species_PatchDistances/` contains, for each species, habitat definition (narrow/broad) and suitability threshold (High/Medium/Low), a set of derived layers:

- a thresholded habitat raster (`.tif`),
- patch polygons (`.shp` + sidecar files),
- nearest-neighbour indices (`*_closest_5.csv`),
- full nearest-neighbour distance lists (`*_distances.rds`), and
- summary statistics (`*_stats.csv`),

plus a combined summary file `AllSpecies_PatchDistanceStats.csv`. Table 2 gives an overview of these outputs.

Table 2: Overview of files and summary statistics in the Species

Species	Definition	Threshold	Raster	Polygons	Nearest	Distances	Stats	number of	Description
Aglais urticae	narrow	High	Yes	Yes	Yes	Yes	Yes	220508	Thresholded habitat raster, p
Aglais urticae	narrow	Medium	Yes	Yes	Yes	Yes	Yes	675440	Thresholded habitat raster, p
Aglais urticae	narrow	Low	Yes	Yes	Yes	Yes	Yes	693840	Thresholded habitat raster, p
Aglais urticae	broad	High	Yes	Yes	Yes	Yes	Yes	503318	Thresholded habitat raster, p
Aglais urticae	broad	Medium	Yes	Yes	Yes	Yes	Yes	568226	Thresholded habitat raster, p
Aglais urticae	broad	Low	Yes	Yes	Yes	Yes	Yes	571597	Thresholded habitat raster, p
Coenonympha tullia	narrow	High	Yes	Yes	Yes	Yes	Yes	2610	Thresholded habitat raster, p
Coenonympha tullia	narrow	Medium	Yes	Yes	Yes	Yes	Yes	7652	Thresholded habitat raster, p
Coenonympha tullia	narrow	Low	Yes	Yes	Yes	Yes	Yes	11855	Thresholded habitat raster, p
Coenonympha tullia	broad	High	Yes	Yes	Yes	Yes	Yes	2610	Thresholded habitat raster, p
Coenonympha tullia	broad	Medium	Yes	Yes	Yes	Yes	Yes	7652	Thresholded habitat raster, p
Coenonympha tullia	broad	Low	Yes	Yes	Yes	Yes	Yes	26131	Thresholded habitat raster, p
Gonepteryx rhamni	narrow	High	Yes	Yes	Yes	Yes	Yes	244615	Thresholded habitat raster, p
Gonepteryx rhamni	narrow	Medium	Yes	Yes	Yes	Yes	Yes	245331	Thresholded habitat raster, p
Gonepteryx rhamni	narrow	Low	Yes	Yes	Yes	Yes	Yes	693651	Thresholded habitat raster, p
Gonepteryx rhamni	broad	High	Yes	Yes	Yes	Yes	Yes	256740	Thresholded habitat raster, p
Gonepteryx rhamni	broad	Medium	Yes	Yes	Yes	Yes	Yes	255971	Thresholded habitat raster, p
Gonepteryx rhamni	broad	Low	Yes	Yes	Yes	Yes	Yes	570815	Thresholded habitat raster, p
Lycaena virgaureae	narrow	High	Yes	Yes	Yes	Yes	Yes	234050	Thresholded habitat raster, p
Lycaena virgaureae	narrow	Medium	Yes	Yes	Yes	Yes	Yes	264563	Thresholded habitat raster, p
Lycaena virgaureae	narrow	Low	Yes	Yes	Yes	Yes	Yes	332385	Thresholded habitat raster, p
Lycaena virgaureae	broad	High	Yes	Yes	Yes	Yes	Yes	234050	Thresholded habitat raster, p
Lycaena virgaureae	broad	Medium	Yes	Yes	Yes	Yes	Yes	264563	Thresholded habitat raster, p
Lycaena virgaureae	broad	Low	Yes	Yes	Yes	Yes	Yes	346530	Thresholded habitat raster, p
Maniola jurtina	narrow	High	Yes	Yes	Yes	Yes	Yes	257648	Thresholded habitat raster, p
Maniola jurtina	narrow	Medium	Yes	Yes	Yes	Yes	Yes	701709	Thresholded habitat raster, p
Maniola jurtina	narrow	Low	Yes	Yes	Yes	Yes	Yes	732306	Thresholded habitat raster, p
Maniola jurtina	broad	High	Yes	Yes	Yes	Yes	Yes	535212	Thresholded habitat raster, p
Maniola jurtina	broad	Medium	Yes	Yes	Yes	Yes	Yes	573501	Thresholded habitat raster, p
Maniola jurtina	broad	Low	Yes	Yes	Yes	Yes	Yes	574934	Thresholded habitat raster, p
Pieris napi	narrow	High	Yes	Yes	Yes	Yes	Yes	243084	Thresholded habitat raster, p
Pieris napi	narrow	Medium	Yes	Yes	Yes	Yes	Yes	622973	Thresholded habitat raster, p
Pieris napi	narrow	Low	Yes	Yes	Yes	Yes	Yes	684203	Thresholded habitat raster, p
Pieris napi	broad	High	Yes	Yes	Yes	Yes	Yes	496107	Thresholded habitat raster, p
Pieris napi	broad	Medium	Yes	Yes	Yes	Yes	Yes	537189	Thresholded habitat raster, p
Pieris napi	broad	Low	Yes	Yes	Yes	Yes	Yes	575560	Thresholded habitat raster, p
Thymelicus lineola	narrow	High	Yes	Yes	Yes	Yes	Yes	255000	Thresholded habitat raster, p
Thymelicus lineola	narrow	Medium	Yes	Yes	Yes	Yes	Yes	269712	Thresholded habitat raster, p
Thymelicus lineola	narrow	Low	Yes	Yes	Yes	Yes	Yes	731056	Thresholded habitat raster, p
Thymelicus lineola	broad	High	Yes	Yes	Yes	Yes	Yes	255000	Thresholded habitat raster, p
Thymelicus lineola	broad	Medium	Yes	Yes	Yes	Yes	Yes	264807	Thresholded habitat raster, p
Thymelicus lineola	broad	Low	Yes	Yes	Yes	Yes	Yes	573722	Thresholded habitat raster, p

2 Overview

This repository documents the workflow used to generate spatial layers for:

- **Project 1 – Contemporary Habitat Configuration and Population Genomics**, and
- the habitat-suitability component that will be reused in **Project 2 – Historical / Temporal Habitat Configuration and Population Museomics**.

The pipeline:

1. Reads **expert-opinion habitat suitability** for butterfly species.
2. Reads a **habitat class mapping** linking conceptual habitats to a national land-use basemap.
3. Maps those habitats to land-use classes via the basemap VAT table.
4. Generates **binary habitat masks per habitat**.

5. Derives **structural layers** from EcoDes data (vegetation density, canopy height).
6. Builds **forest-edge distance** and **forest-gap** layers.
7. Builds **distance-to-coast** masks.
8. Combines habitat masks, structural layers, and expert opinions into species-specific habitat rasters, again for both narrow and broad definitions.

Most outputs are written as Cloud Optimized GeoTIFFs (COGs) for efficient sharing and reuse.

3 Packages

```
library(terra)
library(tidyverse)    # dplyr, tidyr, stringr, etc.
library(foreign)      # read.dbf
library(readxl)
library(janitor)
library(geodata)
library(tidyterra)
library(patchwork)

# Custom / project packages:
library(SpeciesPoolR)
library(BDRUtils)

# New for connectivity / distances
library(FNN)           # nearest neighbours between patches
library(kableExtra)
```

3.1 Data and folder structure

The code assumes the following structure relative to the project root:

- Basemap/lu_00_2021.tif and Basemap/lu_00_2021.tif.vat.dbf
- Majority_Expert_Opinions.xlsx
- Classes_Butterfiles_.xlsx
- Optional EcoDes links:
 - .../vegetation_density.vrt
 - .../canopy_height.vrt

Output folders are created automatically if missing:

- SpeciesHabs/ – one COG per species (*_narrow_.tif, *_broad_.tif)
- Misc derived layers in the working directory:
 - HighDensity.tif, LowDensity.tif, canopy_height_bellow_2m.tif, etc.

4 Input data

This workflow uses two main tabular inputs plus a national land-use basemap with an associated Value Attribute Table (VAT).

4.1 Majority expert opinions

This file (`Majority_Expert_Opinions.xlsx`) stores the expert-opinion habitat suitability for each species. Columns:

- **art** Latin species name (e.g. *Aglais urticae*).
- **Habitat** Text label for the habitat category (e.g. “Grøftekanter (0–3)”) that matches the **habitat** column in `Classes_Butterfiles.xlsx` once the “(0–3)” suffix is removed.
- **Majority** Integer score in the range 0–3, summarizing the experts’ majority score for that species–habitat combination.
- **proportion** Proportion of experts who answered (e.g. 0.6, 0.8). (*Not used in the current pipeline; we rely on Majority*).

In the code, we:

1. Convert **Majority** to numeric.
2. Rescale it to the range 0–1 by dividing by 3 and rounding to 1 decimal place.
3. Filter out combinations with `Majority == 0` (no use of that habitat).

We then use this rescaled **Majority** as a **weight** when combining habitat rasters for each species.

4.2 Habitat–basemap mapping (`Classes_Butterflies`)

This file (`Classes_Butterfiles.xlsx`) links conceptual habitat categories to one or more land-cover classes in the national land-use basemap (`lu_00_2021.tif`).

Columns:

- **habitat** Named habitat category (e.g. *Næringsrig skov*, *Tæt skov*, *Lysåben skov*, *Skovkanter*).
- **Narrow**: A list of “narrow” land-use classes that appear in the VAT columns `C_12`, `C_09`, or `C_05` of `lu_00_2021.tif`. These strings often contain multiple classes separated by dashes/commas; they are cleaned and split so each individual class (e.g. “Birk”, “Fyr”, “Stilkeke-krat”) becomes its own row.
- **Broad** (in the Excel the column is named Broad with a trailing space) A broader / wider grouping of classes. Some rows contain the text “as narrow”, meaning the broad definition is identical to the narrow one. If Broad is missing or “as narrow”, we default to using Narrow here.
- **Additional layers** Optional notes about extra spatial constraints, often referring to ECODES layers, e.g.:
 - ECODES density > 90 %: use **high-density forest** mask
 - ECODES density < 50 %: use **low-density forest** mask
 - % meters of all forest edges / Only include edges e.g. 2 metres on both sides ...: use **forest-edge band**
 - ECODES; areas with hight < 2 metres in forest, and > 20x20 m wihtin forests: use **canopy gaps < 2 m, > 400 square meters**
 - OBS - distance to coast 200 m: use **coast 200 m mask**

These are used to **automatically attach ECODES structural masks** to those habitats (and thus to the species that use them).

- `extracted_text_05`, `extracted_text_09`, `extracted_text_12` Extracted label text corresponding to different VAT columns.

Conceptually:

- `Classes_Butterfiles_.xlsx` says for habitat *H*, include these land-use classes (narrow vs broad), and optionally combine with these ECODES structural layers.
- The VAT table for `lu_00_2021.tif` tells us “which numeric codes correspond to those class labels”.
- We then build **binary habitat rasters**, for both a narrow and broad definition, and later combine them with species-level expert weights.

4.3 How the two tables interact

1. `Classes_Butterfiles_.xlsx` defines how conceptual habitats map to the basemap (`habitat(Narrow / Broad): VALUE` via the VAT), and which habitats should be further constrained by ECODES structural layers.
2. `Majority_Expert_Opinions.xlsx` defines, for each species `art`, which conceptual habitats it uses and with what weight (`Majority`).
3. The **species habitat rasters** in `SpeciesHabs/` are built by:
 - starting from the relevant habitat masks in `Habitats/` (narrow and/or broad),
 - for some habitats, chaining in extra ECODES structural layers (low density, high density, forest edges, canopy gaps, coast 200 m),
 - multiplying each masked habitat by the expert weight `Majority` for that species, and
 - taking the pixel-wise maximum across all contributing habitats for that species.

So species that depend on **low-density forest**, **forest edges**, or **coastal areas** automatically get those structural constraints applied, but only for the relevant habitats.

5 Expert opinions and basemap

```
# Expert opinions (0-3) scaled to 0-1 and filtered
Expert_opinion <- read_excel("Majority_Expert_Opinions.xlsx") |>
  dplyr::mutate(Majority = round(as.numeric(Majority) / 3, 1)) |>
  dplyr::filter(Majority > 0)

# Basemap raster (land-use)
raster_file <- "Basemap/lu_00_2021.tif"
r <- rast(raster_file)

DK <- geodata::gadm("Denmark", level = 0, path = getwd()) |>
  terra::project(terra::crs(r))

# Value Attribute Table (VAT) for the basemap
dbf_data <- read.dbf("Basemap/lu_00_2021.tif.vat.dbf")

# Clean up the C_12 labels: drop digits, trim whitespace
dbf_data$C_12 <- trimws(gsub("[0-9]", "", dbf_data$C_12))
```

6 Map Excel habitat classes to basemap classes (narrow & broad)

We build two long tables:

- Long_narrow: mapping habitat to VAT VALUE codes using the **Narrow** column,
- Long_broad: mapping habitat to VAT VALUE codes using the **Broad** column.

```
class_map_raw <- readxl::read_excel("Classes_Butterfiles_.xlsx")

# Handle possible trailing space in the column name ("Broad " vs "Broad")
if ("Broad " %in% names(class_map_raw) && !"Broad" %in% names(class_map_raw)) {
  class_map_raw <- dplyr::rename(class_map_raw, Broad = `Broad `)
}

class_map <- class_map_raw |>
  dplyr::mutate(
    # Interpret "as narrow" (case-insensitive) in Broad as "use Narrow"
    Broad = ifelse(
      stringr::str_detect(Broad, regex("as narrow", ignore_case = TRUE)),
      Narrow, Broad
    ),
    # If Broad is NA or empty, also fall back to Narrow
    Broad = ifelse(is.na(Broad) | trimws(Broad) == "", Narrow, Broad)
  )

fyr_types <- c(
  "Bjergfyr", "Frans bjergfyr", "Østrigsk fyr", "Skovfyr",
  "Fransk bjergfyr", "Weymouthsfyr", "Østrigsk fyr", "Skovfyr"
)

# Helper to clean and split a given habitat column ("Narrow" or "Broad")
clean_habitat_column <- function(df, col_name) {

  # 1) Do all the string cleaning and splitting
  df_long <- df |>
    dplyr::select(habitat, Narrow = dplyr::all_of(col_name)) |>
    dplyr::mutate(
      # Fix various naming issues
      Narrow = stringr::str_replace_all(Narrow, "Stilkege-krat", "Stilkege krat"),
      Narrow = stringr::str_replace_all(
        Narrow,
        "Elle- og askeskov ved vandløb, søer og væld",
        "Elle og askeskov ved vandløb, søer og væld"
      ),
      Narrow = stringr::str_replace_all(Narrow, "Ege-blandskov", "Ege blandskov"),
      Narrow = stringr::str_replace_all(
        Narrow,
        stringr::fixed("Lav bebyggelse (Low buildings), Have"),
        "Lav bebyggelse"
      ),
      Narrow = stringr::str_replace_all(
        Narrow,
```

```

    stringr::fixed("Overdrev (Slette)"),
    "Slette, Overdrev (Slette)"
  ),
  Narrow = stringr::str_replace_all(
    Narrow,
    stringr::fixed("Overdrev (overdrev)"),
    "Slette, Overdrev (overdrev)"
  ),
  Narrow = stringr::str_replace_all(
    Narrow,
    stringr::fixed("Overdrev (græsset)"),
    "Slette, Overdrev (græsset)"
  ),
  Narrow = stringr::str_replace_all(Narrow, stringr::fixed("Kilder"), "Kildevæld"),
  Narrow = stringr::str_replace_all(Narrow, stringr::fixed("Ruin"), "Ruin, gravhøj"),
  Narrow = stringr::str_replace_all(Narrow, stringr::fixed("Vandløbskant"), "Vandløbskant")
) |>
dplyr::mutate(Narrow = stringr::str_split(Narrow, "-")) |>
tidyr::unnest(Narrow) |>
dplyr::mutate(
  Narrow = stringr::str_trim(Narrow),
  Narrow = stringr::str_remove_all(Narrow, "^,|,$")
) |>
dplyr::filter(
  Narrow != "",
  !is.na(Narrow),
  Narrow != ",",
  Narrow != "NA"
) |>
dplyr::mutate(
  Narrow = stringr::str_replace_all(
    Narrow,
    "Elle og askeskov ved vandløb, søer og væld",
    "Elle- og askeskov ved vandløb, søer og væld"
  ),
  Narrow = stringr::str_replace_all(Narrow, "Stilkege krat", "Stilkege-krat"),
  Narrow = stringr::str_replace_all(Narrow, "Ege blandskov", "Ege-blandskov")
)

# 2) Expand generic "Fyr" to all specific Fyr types
Fyr_expanded <- df_long |>
dplyr::filter(Narrow == "Fyr") |>
dplyr::select(habitat) |>
tidyr::uncount(length(fyr_types)) %>%
dplyr::mutate(
  Narrow = rep(fyr_types, times = nrow(.) / length(fyr_types))
)

# 3) Replace generic "Fyr" rows with the expanded ones
df_long <- df_long |>
dplyr::filter(Narrow != "Fyr") |>
dplyr::bind_rows(Fyr_expanded)

```

```

df_long
}

Long_narrow <- clean_habitat_column(class_map, "Narrow")
Long_broad  <- clean_habitat_column(class_map, "Broad")

```

Now we identify which VAT column (C_12, C_09, C_05) each class belongs to and assign the corresponding VALUE code, separately for narrow and broad.

```

add_vat_values <- function(Long, dbf) {
  Long$layer <- NA_character_
  Long$value <- NA_integer_

  Long$layer[Long$Narrow %in% dbf$C_12] <- "C_12"
  Long$layer[Long$Narrow %in% dbf$C_09] <- "C_09"
  Long$layer[Long$Narrow %in% dbf$C_05] <- "C_05"

  NotHereYet <- Long |>
    dplyr::filter(is.na(layer)) |>
    dplyr::pull(Narrow) |>
    unique()

  if (length(NotHereYet) > 0) {
    message("Classes not found in VAT: ",
            paste(NotHereYet, collapse = ", "))
  }

  Long <- Long |>
    dplyr::filter(!is.na(layer))

  for (i in seq_len(nrow(Long))) {
    col_i <- Long$layer[i]
    val_i <- Long$Narrow[i]
    Long$value[i] <- dbf$VALUE[dbf[[col_i]] == val_i]
  }

  Long
}

Long_narrow <- add_vat_values(Long_narrow, dbf_data)
Long_broad  <- add_vat_values(Long_broad,  dbf_data)

HABS <- sort(unique(class_map$habitat))
dir.create("Habitats", showWarnings = FALSE)

```

6.1 Build habitat masks (COGs) per habitat (narrow & broad)

For each habitat, we build two binary rasters:

- *_narrow.tif → based on the **Narrow** definition.
- *_broad.tif → based on the **Broad** (“wide”) definition.

```

for (hab_i in HABS) {

  # NARROW definition
  Temp_narrow <- Long_narrow |>
    dplyr::filter(habitat == hab_i) |>
    dplyr::pull(value)

  if (length(Temp_narrow) > 0) {
    NewRast_narrow <- terra::ifel(as.numeric(r) %in% Temp_narrow, 1, 0)

    out_narrow <- paste0(
      "Habitats/",
      janitor::make_clean_names(hab_i),
      "_narrow.tif"
    )

    SpeciesPoolR::write_cog(NewRast_narrow, out_narrow)
  }

  # BROAD (wide) definition
  Temp_broad <- Long_broad |>
    dplyr::filter(habitat == hab_i) |>
    dplyr::pull(value)

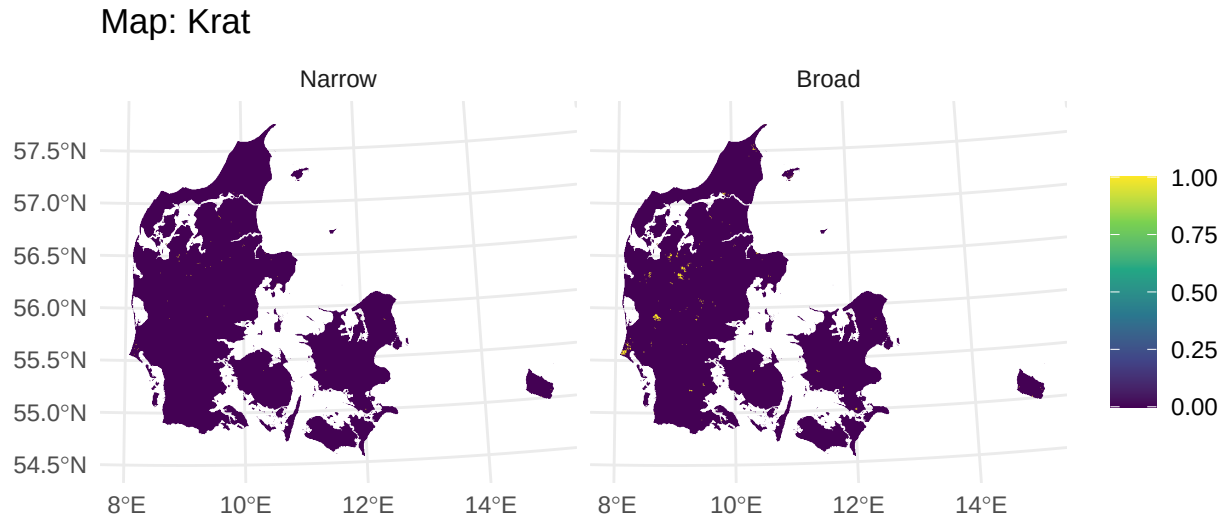
  if (length(Temp_broad) > 0) {
    NewRast_broad <- terra::ifel(as.numeric(r) %in% Temp_broad, 1, 0)

    out_broad <- paste0(
      "Habitats/",
      janitor::make_clean_names(hab_i),
      "_broad.tif"
    )

    SpeciesPoolR::write_cog(NewRast_broad, out_broad)
  }
}

```

6.1.1 Quick visual check



7 EcoDes structural layers: vegetation density and canopy height

We derive:

- a high-density and low-density forest mask from EcoDes vegetation density, and
- a canopy < 2 m mask from EcoDes canopy height.

```
terraOptions(memfrac = 0.8, tempdir = "D:/WD_R/temp")
Sys.setenv(GDAL_NUM_THREADS = 10)
```

```
Density <- rast(
  "o:/Nat_Ecoinformatics/B_Read/Denmark/DK_EcoDes/EcoDes-DK15_v1.1.0/vegetation_density/vegetation_density.tif",
  srcargs = list(
    -v
  )
)

thr <- app(
  Density,
  fun = function(x) cbind(as.integer(x > 9000L), as.integer(x < 5000L)),
  filename = "density_thresholds.tif",
  overwrite = TRUE,
  cores = max(1, parallel::detectCores() / 2),
  wopt = list(
    -v
  )
)
```

```

names      = c("HighDensity", "LowDensity"),
datatype   = "INT1U",
gdal       = c(
  "COMPRESS=ZSTD", "TILED=YES",
  "BLOCKXSIZE=512", "BLOCKYSIZE=512",
  "NUM_THREADS=ALL_CPUS", "BIGTIFF=IF_SAFER"
)
)
)

BDRUtils::write_cog(thr[[1]], "HighDensity.tif")
BDRUtils::write_cog(thr[[2]], "LowDensity.tif")

```

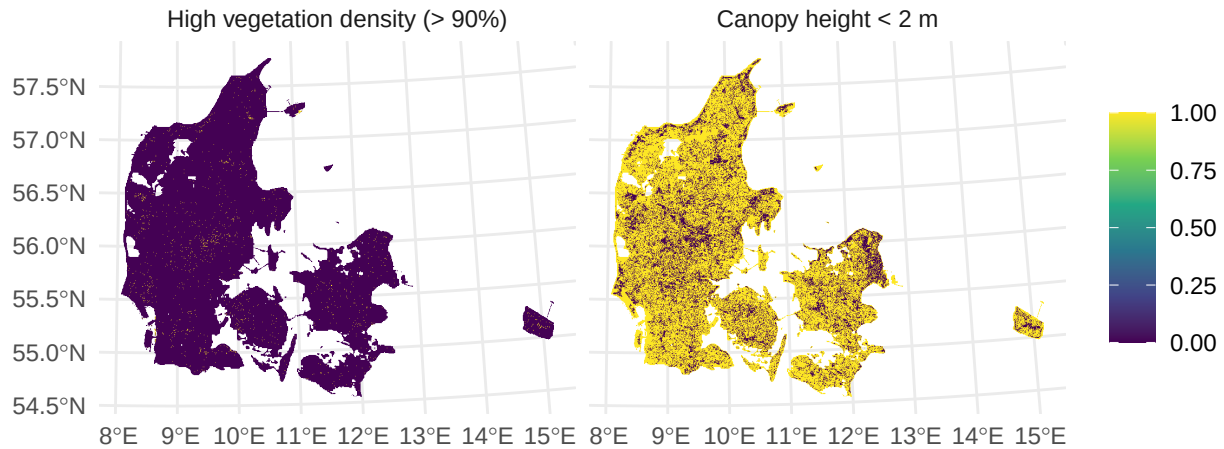
```

canopy_height <- rast(
  "o:/Nat_Ecoinformatics/B_Read/Denmark/DK_EcoDes/EcoDes-DK15_v1.1.0/canopy_height/canopy_height.vrt"
)

thr_canopy <- app(
  canopy_height,
  fun = function(x) cbind(as.integer(x < 200L)),
  filename = "canopy_height_bellow_2m.tif", # note: filename kept as in original code
  overwrite = TRUE,
  cores = max(1, parallel::detectCores() / 2),
  wopt = list(
    names      = c("canopy_height_bellow_2m"),
    datatype   = "INT1U",
    gdal       = c(
      "COMPRESS=ZSTD", "TILED=YES",
      "BLOCKXSIZE=512", "BLOCKYSIZE=512",
      "NUM_THREADS=ALL_CPUS", "BIGTIFF=IF_SAFER"
    )
  )
)
)

```

7.0.1 Quick visual check



8 Forest edge distance and 200 m forest border

We use a forest mask (`Habitats/skovkanter_narrow.tif`) to compute:

- distance to the nearest forest edge for every pixel, and
- a 200 m band on both sides of the forest edge.

```
# Forest edge / forest mask
skovkanterv_narrow <- rast("Habitats/skovkanter_narrow.tif")

x <- skovkanterv_narrow

# Sources for distance: opposite classes
src_nonforest <- ifel(x == 0, 1, NA) # distance FROM forest cells TO nearest non-forest
src_forest    <- ifel(x == 1, 1, NA) # distance FROM non-forest cells TO nearest forest

d_to_nonforest <- distance(
  src_nonforest,
  filename = "d_to_nonforest.tif", overwrite = TRUE,
  wopt = list(gdal = c("COMPRESS=LZW", "TILED=YES", "BIGTIFF=YES"))
)
```

```

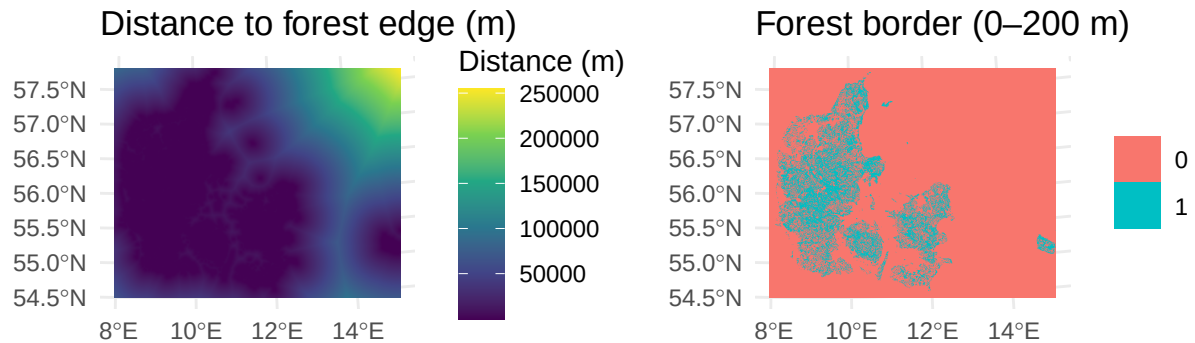
d_to_forest <- distance(
  src_forest,
  filename = "d_to_forest.tif", overwrite = TRUE,
  wopt = list(gdal = c("COMPRESS=LZW", "TILED=YES", "BIGTIFF=YES"))
)

# Distance to the edge = distance to the opposite class
dist_edge <- ifel(x == 1, d_to_nonforest, d_to_forest)
writeRaster(
  dist_edge,
  "dist_to_edge.tif",
  overwrite = TRUE,
  wopt = list(gdal = c("COMPRESS=LZW", "TILED=YES", "BIGTIFF=YES"))
)

# 200 m band on both sides of the edge
edge200 <- dist_edge <= 200
writeRaster(
  edge200,
  "forest_border_200m.tif",
  overwrite = TRUE,
  wopt = list(
    datatype = "INT1U",
    gdal      = c("COMPRESS=LZW", "TILED=YES")
  )
)

```

8.0.1 Quick visual check



9 Forest gaps: canopy < 2 m inside forest holes > 400 m²

We identify low-canopy patches inside “holes” in forest (clearings) larger than 400 m², then rasterize them.

```
# Canopy < 2 m (1/NA)
canopy_bellow_2 <- rast("canopy_height_bellow_2m.tif")

# Forest raster (assumed 1 = forest, NA or 0 = non-forest)
forestEdge <- rast("Habitats/skovkanter_narrow.tif")

# Forest polygons, split into individual parts
forestEdgePoly <- terra::as.polygons(forestEdge) |>
  terra::disagg()

# Holes inside forest polygons
ForestHoles <- terra::fillHoles(forestEdgePoly, inverse = TRUE)

# Keep holes > 400 m² (~20 x 20 m)
ForestHolesOver400sqmt <- ForestHoles[terra::expanses(ForestHoles) > 400, ]

# Rasterize holes onto canopy grid
ForestHolesOver400sqmtRast <- terra::rasterize(
```

```

ForestHolesOver400sqmt,
canopy_bellow_2,
field = 1
)

# Combine low canopy and holes
HolesWithLowCanopy <- canopy_bellow_2 + ForestHolesOver400sqmtRast

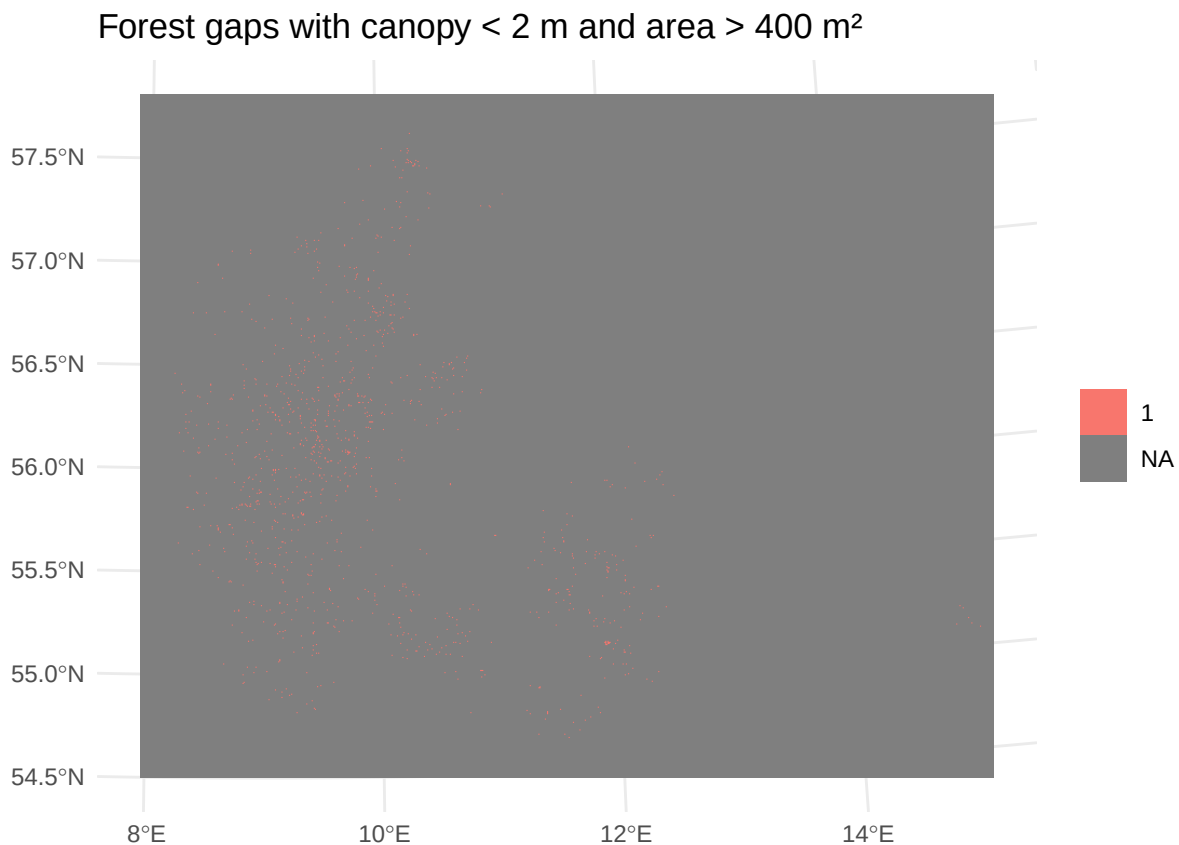
# Keep only pixels where both are 1 (sum == 2)
HolesWithLowCanopy <- terra::ifel(HolesWithLowCanopy == 2, 1, NA)

# Convert to polygons and filter by size again
HolesWithLowCanopy <- terra::as.polygons(HolesWithLowCanopy) |>
  terra::disagg()
HolesWithLowCanopy <- HolesWithLowCanopy[terra::expansion(HolesWithLowCanopy) > 400, ]

forest_gaps_canopy_lt2m_400m2 <- terra::rasterize(HolesWithLowCanopy, forestEdge)
BDRUtils::write_cog(forest_gaps_canopy_lt2m_400m2, "forest_gaps_canopy_lt2m_400m2.tif")

```

9.0.1 Quick visual check



10 Distance-to-coast masks (200, 400, 800 m)

We create distance-to-coast rasters and then binary masks within 200, 400, and 800 m of the coast.

```
# Denmark boundary in same CRS as basemap r
DK <- geodata::gadm("Denmark", level = 0, path = getwd()) |>
  terra::project(terra::crs(r))

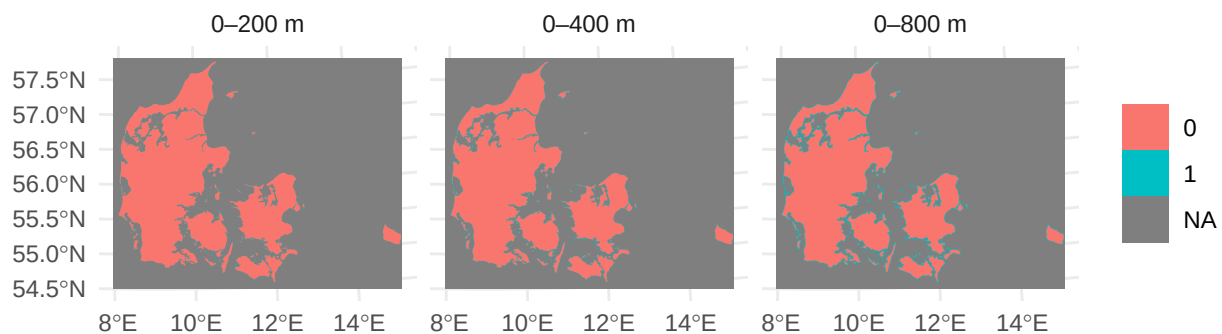
DK_Coast <- terra::rasterize(DK, r, field = NA, background = 1)
DK_Coast_m <- terra::distance(DK_Coast)
DK_Coast_m <- terra::mask(DK_Coast_m, DK)

DK_Coast_200m <- terra::ifel(DK_Coast_m <= 200, 1, 0)
BDRUtils::write_cog(DK_Coast_200m, "DK_Coast_200m.tif")

DK_Coast_400m <- terra::ifel(DK_Coast_m <= 400, 1, 0)
BDRUtils::write_cog(DK_Coast_400m, "DK_Coast_400m.tif")

DK_Coast_800m <- terra::ifel(DK_Coast_m <= 800, 1, 0)
BDRUtils::write_cog(DK_Coast_800m, "DK_Coast_800m.tif")
```

10.0.1 Quick visual check



11 Species-specific habitat rasters (narrow & broad, with structural layers)

Here we combine:

- **Habitat masks** (narrow & broad) from `Habitats/`,
- **ECODES structural layers** for the habitats that need them (as specified in `Additional layers`), and
- **Expert weights** (Majority) from `Majority_Expert_Opinions.xlsx`,

to generate **one or two rasters per species**:

- `<species>_narrow.tif`
- `<species>_broad.tif`

11.1 Set up structural flags per habitat

We translate the text in `Additional layers` into simple flags telling us which structural rasters to apply to each habitat.

```
# Flags: for each habitat, which structural layers should be used?
hab_struct <- class_map |>
  dplyr::mutate(
    # avoid NA issues: turn NA into ""
    Additional_clean = dplyr::coalesce(`Additional layers`, "")
  ) |>
  dplyr::transmute(
    habitat,
    add_highdensity = stringr::str_detect(Additional_clean, "density > 90"),
    add_lowdensity  = stringr::str_detect(Additional_clean, "density < 50"),
    add_edges200    = stringr::str_detect(
      Additional_clean,
      stringr::regex("forest edges|edges e.g. 2 metres", ignore_case = TRUE)
    ),
    add_gaps_lt2    = stringr::str_detect(
      Additional_clean,
      stringr::regex("height < 2 metres", ignore_case = TRUE)
    ),
    add_coast200    = stringr::str_detect(
      Additional_clean,
      stringr::regex("distance to coast 200", ignore_case = TRUE)
    )
  )

# Load structural rasters (assumed to be on the same grid as lu_00_2021.tif)
temp <- terra::rast("Habitats/avneknippemose_narrow.tif")
HighDensity <- rast("HighDensity.tif") |>
  terra::resample(temp, method = "near")
LowDensity <- rast("LowDensity.tif") |>
  terra::resample(temp, method = "near")
```

```

Edge200      <- rast("forest_border_200m.tif") |>
  terra::resample(temp, method = "near")
Gaps_lt2     <- rast("forest_gaps_canopy_lt2m_400m2.tif") |>
  terra::resample(temp, method = "near")

Gaps_lt2 <- ifel(is.na(Gaps_lt2), 0, Gaps_lt2)

Coast200     <- rast("DK_Coast_200m.tif") |>
  terra::resample(temp, method = "near")

struct_layers <- list(
  HighDensity = HighDensity,
  LowDensity  = LowDensity,
  Edge200     = Edge200,
  Gaps_lt2    = Gaps_lt2,
  Coast200    = Coast200
)

# Helper: apply the structural masks that are relevant for a given habitat label
apply_struct_masks <- function(r_hab, hab_label, hab_struct, struct_layers) {
  row <- hab_struct[hab_struct$habitat == hab_label, , drop = FALSE]

  if (nrow(row) == 0) return(r_hab)

  out <- r_hab

  if (isTRUE(row$add_highdensity)) out <- out * struct_layers$HighDensity
  if (isTRUE(row$add_lowdensity))  out <- out * struct_layers$LowDensity
  if (isTRUE(row$add_edges200))    out <- out * struct_layers$Edge200
  if (isTRUE(row$add_gaps_lt2))    out <- out * struct_layers$Gaps_lt2
  if (isTRUE(row$add_coast200))    out <- out * struct_layers$Coast200

  out
}

```

11.2 Build species rasters (narrow + broad)

```

## Build species rasters (narrow + broad), disk-friendly

dir.create("SpeciesHabs", showWarnings = FALSE)

SPP <- unique(Expert_opinion$art)

# Clean up Habitat labels in Expert_opinion so they match class_map$habitat
Expert_opinion$Habitat <- trimws(
  gsub("\\s*\\(0-3\\)", "", Expert_opinion$Habitat)
)

# Optional: tell terra to favour disk
terraOptions(memfrac = 0.6, todisk = TRUE)

```

```

for (i in seq_along(SPP)) {
  spp_name <- SPP[i]
  spp_id   <- janitor::make_clean_names(spp_name)

  message("==== Starting species ", i, ": ", spp_name, " @ ", Sys.time(), " =====")

  Species <- Expert_opinion |>
    dplyr::filter(art == spp_name)

  # temp folder for per-habitat rasters of this species
  tmp_dir <- file.path("SpeciesHabs", "tmp", spp_id)
  dir.create(tmp_dir, recursive = TRUE, showWarnings = FALSE)

  tmp_narrow_files <- character(0)
  tmp_broad_files  <- character(0)

  for (j in seq_len(nrow(Species))) {
    hab_label <- Species$Habitat[j]
    hab_weight <- Species$Majority[[j]]
    hab_clean <- janitor::make_clean_names(hab_label)

    # Paths to habitat rasters
    path_narrow <- file.path("Habitats", paste0(hab_clean, "_narrow.tif"))
    path_broad  <- file.path("Habitats", paste0(hab_clean, "_broad.tif"))

    ## ----- NARROW version -----
    if (file.exists(path_narrow)) {
      r_hab_n <- try(terra::rast(path_narrow), silent = TRUE)

      if (inherits(r_hab_n, "SpatRaster")) {
        # Apply structural masks if this habitat has associated ECODES constraints
        r_hab_n <- apply_struct_masks(r_hab_n, hab_label, hab_struct, struct_layers)

        # Apply expert weight
        r_weighted_n <- r_hab_n * hab_weight

        # Write to a temp file for this species
        tmp_file_n <- file.path(tmp_dir, sprintf("hab_%02d_narrow.tif", j))
        terra::writeRaster(
          r_weighted_n,
          tmp_file_n,
          overwrite = TRUE,
          wopt = list(
            datatype = "FLT4S",
            gdal      = c("COMPRESS=ZSTD", "TILED=YES", "BIGTIFF=IF_SAFER")
          )
        )

        tmp_narrow_files <- c(tmp_narrow_files, tmp_file_n)

        message("NARROW habitat ", j, " of ", nrow(Species),
          " written @ ", Sys.time())
      } else {

```

```

    message("Skipping narrow habitat ", j, " (not a valid raster)")
  }
} else {
  message("Missing narrow file for habitat ", j, " - ", path_narrow)
}

## ----- BROAD version -----
if (file.exists(path_broad)) {
  r_hab_b <- try(terra::rast(path_broad), silent = TRUE)

  if (inherits(r_hab_b, "SpatRaster")) {
    r_hab_b <- apply_struct_masks(r_hab_b, hab_label, hab_struct, struct_layers)
    r_weighted_b <- r_hab_b * hab_weight

    tmp_file_b <- file.path(tmp_dir, sprintf("hab_%02d_broad.tif", j))
    terra::writeRaster(
      r_weighted_b,
      tmp_file_b,
      overwrite = TRUE,
      wopt = list(
        datatype = "FLT4S",
        gdal      = c("COMPRESS=ZSTD", "TILED=YES", "BIGTIFF=IF_SAFER")
      )
    )

    tmp_broad_files <- c(tmp_broad_files, tmp_file_b)

    message("BROAD habitat ", j, " of ", nrow(Species),
            " written @ ", Sys.time())
  } else {
    message("Skipping broad habitat ", j, " (not a valid raster)")
  }
} else {
  message("Missing broad file for habitat ", j, " - ", path_broad)
}

} # end inner loop over habitats

## ----- Mosaic TEMP rasters + final species rasters -----

# NARROW mosaic + mask
if (length(tmp_narrow_files) > 0) {
  r_list_n <- lapply(tmp_narrow_files, terra::rast)

  out_narrow <- file.path(
    "SpeciesHabs",
    paste0(spp_id, "_narrow_.tif")
  )

  # Pixel-wise max across all narrow habitat rasters
  Final_narrow <- do.call(
    terra::mosaic,
    c(

```

```

    r_list_n,
    fun      = "max",
    filename = out_narrow,
    overwrite = TRUE
  )
)

# Mask by DK directly on disk (overwriting the same file)
Final_narrow <- terra::mask(
  Final_narrow,
  DK,
  filename = out_narrow,
  overwrite = TRUE
)

message("NARROW species raster written: ", out_narrow, " @ ", Sys.time())
} else {
  message("No valid NARROW rasters found for ", spp_name)
}

# BROAD mosaic + mask
if (length(tmp_broad_files) > 0) {
  r_list_b <- lapply(tmp_broad_files, terra::rast)

  out_broad <- file.path(
    "SpeciesHabs",
    paste0(spp_id, "_broad_.tif")
  )

  Final_broad <- do.call(
    terra::mosaic,
    c(
      r_list_b,
      fun      = "max",
      filename = out_broad,
      overwrite = TRUE
    )
  )

  Final_broad <- terra::mask(
    Final_broad,
    DK,
    filename = out_broad,
    overwrite = TRUE
  )

  message("BROAD species raster written: ", out_broad, " @ ", Sys.time())
} else {
  message("No valid BROAD rasters found for ", spp_name)
}

## ----- Clean up temp stuff for this species -----

```

```

unlink(tmp_dir, recursive = TRUE, force = TRUE)
terra::tmpFiles(current = TRUE, remove = TRUE)
gc()

message("==== Finished species ", spp_name, " @ ", Sys.time(), " =====")
}

```

11.2.1 Quick visual check for one species

```

SPP <- unique(Expert_opinion$art)

if (length(SPP) > 0) {
  spp_example <- SPP[3]
  spp_id      <- janitor::make_clean_names(spp_example)

  file_n <- file.path("SpeciesHabs", paste0(spp_id, "_narrow.tif"))
  file_b <- file.path("SpeciesHabs", paste0(spp_id, "_broad.tif"))

  Narrow <- rast(file_n)

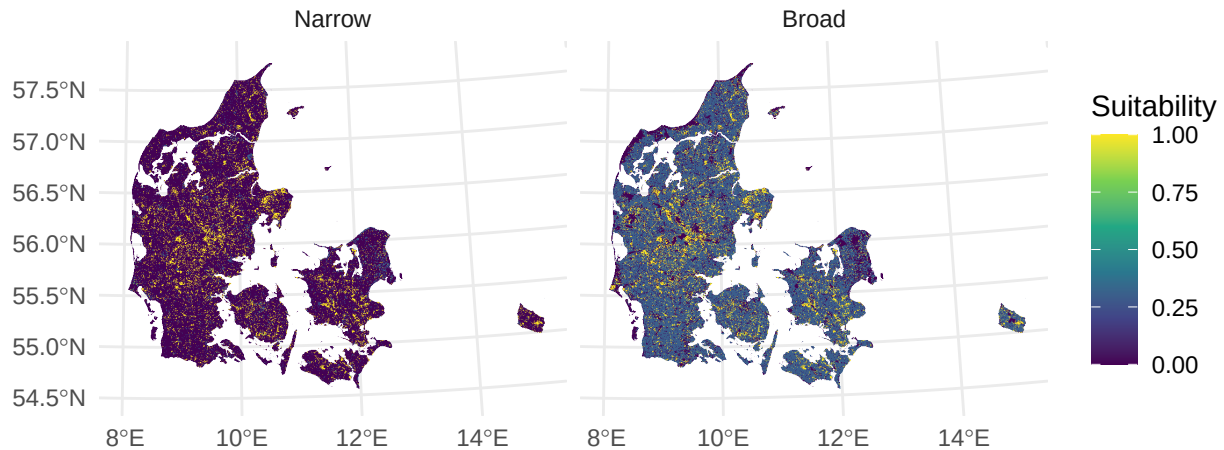
  Broad <- rast(file_b)

  spp_stack <- c(Narrow, Broad)
  names(spp_stack) <- c("Narrow", "Broad")

  ggplot() +
    geom_spatraster(data = spp_stack) +
    scale_fill_viridis_c(na.value = NA) +
    facet_wrap(~lyr) +
    labs(
      title = paste("Habitat suitability (narrow vs broad):", spp_example),
      x = NULL, y = NULL,
      fill = "Suitability"
    ) +
    theme_minimal()
}

```

Habitat suitability (narrow vs broad): *Gonepteryx rhamni*



12 Patch-based connectivity metrics from species rasters

From the species-level habitat suitability rasters in `SpeciesHabs/` we derive patch-based connectivity metrics.

For each species and for both the **narrow** and **broad** habitat definitions, we:

1. Threshold the habitat suitability raster at three values: - **High**: > 0.9
- **Medium**: > 0.6
- **Low**: > 0.2
2. Convert the thresholded raster to polygons (habitat patches).
3. Use k-nearest neighbours (via centroid coordinates and `FNN::get.knn`) to identify, for each patch, its closest neighbours, and then compute polygon–polygon distances between those patches with `terra::distance`.
4. Save: - the thresholded raster - the patch polygons, and - per-species, per-definition, per-threshold summary statistics (mean and median nearest-neighbour distances).

All outputs go into a dedicated folder `Species_PatchDistances/`, and results explicitly distinguish **narrow** vs **broad** species rasters.

```
# Patch-based nearest-neighbour distances for all species  
# using both narrow and broad habitat rasters
```

```
spp_files <- list.files(  
  path = "SpeciesHabs",
```

```

pattern = "\\\\.tif$",
full.names = TRUE
)

out_dir <- "Species_PatchDistances"
dir.create(out_dir, showWarnings = FALSE)

# Thresholds and labels
thresholds <- c(
  High = 0.9,
  Medium = 0.6,
  Low = 0.2
)

all_stats <- list()

for (level in seq_along(thresholds)) {
  level_name <- names(thresholds)[level]
  thr_value <- thresholds[level]

  message("Starting processing for ", level_name,
    " threshold (>", thr_value, ") @ ", Sys.time())

  for (i in seq_along(spp_files)) {

    fpath <- spp_files[i]
    fbase <- basename(fpath)

    # Identify narrow vs broad from filename: *_narrow_.tif / *_broad_.tif
    definition <- dplyr::case_when(
      stringr::str_detect(fbase, "_narrow_") ~ "narrow",
      stringr::str_detect(fbase, "_broad_") ~ "broad",
      TRUE ~ "unknown"
    )

    species_id <- fbase |>
      stringr::str_remove("_narrow_.tif$") |>
      stringr::str_remove("_broad_.tif$")

    species_label <- paste0(species_id, "_", definition)

    message(" Processing ", species_label,
      " (", i, " of ", length(spp_files), ") @ ", Sys.time())

    # --- 1) Thresholded raster and polygons ---

    density_filename <- file.path(
      out_dir,
      paste0(species_label, "_", level_name, ".tif")
    )

    polygon_filename <- file.path(
      out_dir,

```

```

    paste0(species_label, "_", level_name, ".shp")
  )

  if (!file.exists(polygon_filename)) {
    message("    Polygonizing ", level_name, " layer @ ", Sys.time())

    Spp_temp <- terra::rast(fpath)
    # binary mask for this suitability threshold
    layer_bin <- terra::ifel(Spp_temp > thr_value, 1, 0)

    # write the thresholded raster (handy for inspection)
    terra::writeRaster(
      layer_bin, density_filename,
      overwrite = TRUE,
      wopt = list(
        datatype = "INT1U",
        gdal      = c("COMPRESS=ZSTD", "TILED=YES", "BIGTIFF=IF_SAFER")
      )
    )

    # polygons of suitable habitat patches
    layer_polygons <- terra::as.polygons(layer_bin) |>
      terra::disagg()

    terra::writeVector(layer_polygons, polygon_filename, overwrite = TRUE)
  } else {
    message("    Polygon file already exists, reusing @ ", Sys.time())
    layer_polygons <- terra::vect(polygon_filename)
  }

  n_patches <- nrow(layer_polygons)

  # If fewer than 2 patches, distances don't make sense
  if (n_patches < 2) {
    message("    < 2 patches; skipping distance stats for ",
      species_label, " ", level_name)
    next
  }

  # --- 2) Nearest neighbours based on centroids ---

  closest_filename <- file.path(
    out_dir,
    paste0(species_label, "_", level_name, "_closest_5.csv")
  )

  if (!file.exists(closest_filename)) {
    message("    Calculating nearest neighbours @ ", Sys.time())

    coords <- terra::crds(terra::centroids(layer_polygons))

    # Use up to 5 neighbours, but not more than n_patches - 1
    k_use <- max(1, min(5, n_patches - 1))
  }

```

```

nn <- FNN::get.knn(coords, k = k_use)
nn_index <- as.data.frame(nn$nn.index)

readr::write_csv(nn_index, closest_filename)
} else {
  message("    Nearest-neighbour indices already exist; reading")
  nn_index <- readr::read_csv(
    closest_filename,
    col_names = FALSE,
    show_col_types = FALSE
  )
}

# --- 3) Distances and summary stats ---

rds_filename <- file.path(
  out_dir,
  paste0(species_label, "_", level_name, "_distances.rds")
)
csv_filename <- file.path(
  out_dir,
  paste0(species_label, "_", level_name, "_stats.csv")
)

if (!file.exists(csv_filename)) {
  message("    Calculating distance stats for ", level_name,
    " @ ", Sys.time())

  all_distances <- lapply(seq_len(nrow(nn_index)), function(row_i) {
    ref <- layer_polygons[row_i, ]
    others <- layer_polygons[as.integer(nn_index[row_i, ]), ]
    as.numeric(terra::distance(ref, others))
  })

  saveRDS(all_distances, rds_filename)

  all_distances_vec <- unlist(all_distances, use.names = FALSE)

  stats_df <- data.frame(
    Species      = species_id,
    Definition    = definition,          # narrow vs broad
    Threshold    = level_name,          # High / Medium / Low
    ThresholdVal = thr_value,
    n_patches    = n_patches,
    Median       = stats::median(all_distances_vec, na.rm = TRUE),
    Mean         = base::mean(all_distances_vec, na.rm = TRUE)
  )

  readr::write_csv(stats_df, csv_filename)

  key <- paste(species_label, level_name, sep = "_")
  all_stats[[key]] <- stats_df
} else {

```

```
      message("    Stats file already exists; skipping")
    }
  }
}

if (length(all_stats) > 0) {
  all_stats_df <- dplyr::bind_rows(all_stats)
  readr::write_csv(
    all_stats_df,
    file.path(out_dir, "AllSpecies_PatchDistanceStats.csv")
  )
}
```